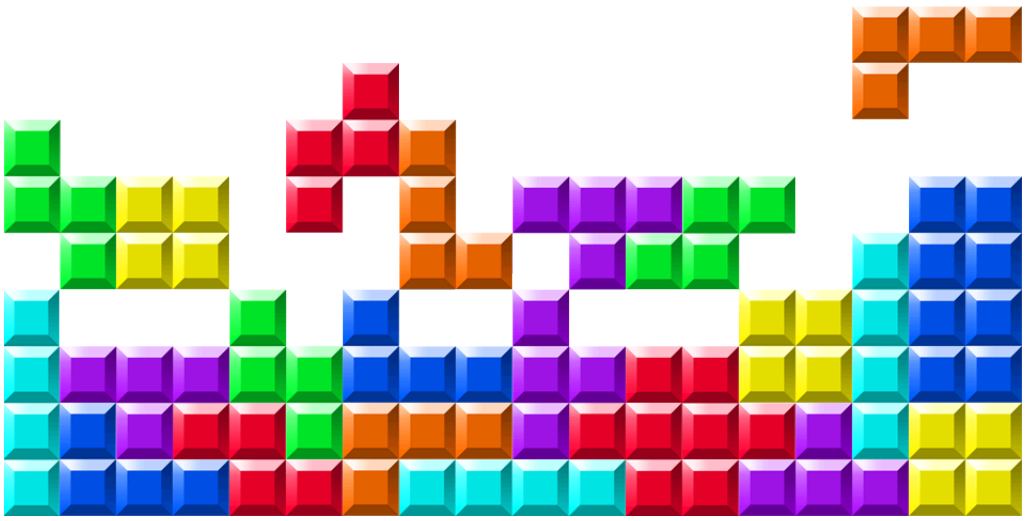
LINFO2275
Second project

Group 2:

Gavin Hope 09022500 – **SINF**
Jakob Thibodeau 12752500 – **DATI**
Lore Lernoux 77562000 – **MAP**



UCLouvain

Professor: Marco Saerens

epl

1 Introduction

2 Theoretical Background

2.1 Q-learning

Q-learning is a model-free reinforcement learning algorithm for learning optimal behaviour through interaction with an environment. It is model-free because it does not require a model of the environment, such as the transition probabilities or reward function. Instead, the agent learns from experience, by trying actions and observing the obtained rewards. In the standard formulation, the environment is modelled as a Markov decision process (MDP) defined by:

- a set of states $\mathcal{S}$,
- a set of actions $\mathcal{A}$,
- a transition probability distribution $p(s', r \mid s, a)$ that gives the probability of transitioning to state s' and receiving reward r after taking action a in state s,
- a discount factor $\gamma \in [0, 1]$ that determines the importance of future rewards.

At time t, the agent observes a state S_t, selects an action A_t, receives a reward R_{t+1}, and moves to a new state S_{t+1}. The objective is to learn a policy that maximises the expected return, where the return is the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The discount factor γ controls how much the agent values future rewards. If γ is close to zero, the agent mainly considers immediate rewards. If γ is close to one, the agent gives more importance to long-term consequences.

The agent learns by estimating action-values, also called Q-values. A Q-value represents how good it is to take a given action in a given state. More precisely, for a policy π, the action-value function is defined as

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right],$$

which is the expected return obtained by taking action a in state s and then following policy π. The goal of Q-learning is to learn the optimal action-value function

$$q_*(s, a) = \max_\pi q_\pi(s, a),$$

which gives the best expected return achievable from each state-action pair. Once this function is known, the optimal action in a state can be chosen greedily:

$$\pi_*(s) \in \arg\max_a q_*(s, a).$$

The optimal Q-values satisfy the Bellman optimality equation:

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

This equation expresses a recursive idea: the value of taking action a in state s is the immediate reward plus the discounted value of the best possible action in the next state. Q-learning uses this idea directly, but without needing to know the transition probabilities. Instead of computing the expectation exactly, it updates the Q-values from sampled transitions experienced by the agent [SB18].

In the tabular version of Q-learning, the agent stores one value $Q(s, a)$ for each state-action pair. After observing a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

the Q-value is updated as follows:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S$$

where $\alpha \in (0, 1]$ is the learning rate. It controls the size of this correction: a large value makes learning faster but less stable, while a small value makes learning slower but smoother.

A common way to select actions during learning is the ε-greedy strategy. With probability $1 - \varepsilon$, the agent chooses the action with the highest current Q-value:

$$A_t \in \arg\max_a Q(S_t, a).$$

With probability ε, it chooses a random action. This allows the agent to balance exploitation and exploration. Exploitation means using the current knowledge to choose the best-known action, while exploration means trying other actions in order to discover potentially better strategies. In practice, ε is often decreased during training, so that the agent explores more at the beginning and exploits more once it has learned useful Q-values [Gee25].

Q-learning is an off-policy algorithm. This means that the policy used to choose actions during training does not need to be the same as the policy being learned. For example, the agent may behave according to an ε-greedy policy in order to explore, while the update itself uses the greedy action in the next state through the term

$$\max_{a'} Q(S_{t+1}, a').$$

Thus, Q-learning learns the value of the greedy policy even while following an exploratory behaviour policy. This is one of the main differences between Q-learning and on-policy methods such as SARSA [SB18].

Overall, Q-learning is a fundamental reinforcement learning algorithm because it shows how an agent can learn optimal behaviour directly from interaction.

3 Implementation

In this project, the Tetris environment is modeled as a Markov Decision Process (MDP) and solved using Deep Q-Networks (DQN). The implementation bridges the gap between raw game states and the reinforcement learning framework. The game and implementation of DQN are extensions of the work of Viet Nguyen [Ngu03].

3.1 Environment

Tetris is a game where blocs of differents shapes and sizes appear at the top of a grid board and fall with time. The goal is to place as many blocks as possible without filling in the whole grid. If an entire row is filled when placing a block, that entire row disappears, and the player scores points. The highest number of rows that can be removed at once is 4, and doing so is called a *Tetris*.

GAVIN YOU HOULD CHANGE THIS PART The players score points every time a block is placed according to the following formula:

$$\text{score} = 1 + 10 \cdot \text{lines\_cleared}^2$$

This score acts as the reward function for the Q-learning models. A penalty of -2 is applied if the `gameover` state is reached to have the agent learn to prolong the game as much as possible.

3.2 Action Space

Unlike traditional Tetris where moving pieces is possible while they fall, the agents have access to a *simplified action space*. An action a is defined as the tuple (x, r), where x is the column where the piece should be dropped and r is the number of rotation of the piece. Since the grid is 10 units wide and a piece can be rotated at most 4 times, the action space contains 40 different actions.

3.3 Deep Q-Learning (DQN)

In complex environments like Tetris, the state space $\mathcal{S}$ is too large and therefore a traditional Q-value table would be very sparse and hard to learn from. Instead, Deep Q-Learning approximates $q_*(s, a)$ using a neural network $Q(s, a; \theta)$, where θ represents the weights of the network.

In this implementation, two distinct neural architectures are used: Neural Networks and Convolutional Neural Networks.

- **Convolutional Neural Network (CNN):** This architecture is used to process the 20×10 binary grid as an image. It utilizes three convolutional layers to extract spatial features and geometric patterns from the Tetris board, followed by dense layers to output the predicted Q-value.

- **Neural Network:** This model consists of three fully connected (linear) layers. It is designed to process a 4-dimensional *smart* feature state vector [`lines, holes, bumpiness, height`] (see

3.4). The layers use ReLU activation functions to introduce non-linearity, mapping the four features to a single scalar Q-value.

To stabilize training of these networks, the networks use **Experience Replay**. Rather than updating the network weights θ using only the most recent transition $(S_t, A_t, R_{t+1}, S_{t+1})$, the model keeps experiences in a replay memory $\mathcal{D}$. During the training step, a mini-batch of transitions is sampled from $\mathcal{D}$ to perform a gradient descent update on the Mean Squared Error (MSE) loss:

$$L(\theta) = \mathbb{E}\left[(y_i - Q(s, a; \theta))^2\right],$$

where the target y_i is derived from the Bellman equation: $y_i = R + \gamma \max_{a'} Q(s', a'; \theta)$.

Furthermore, the ε variable (of the ε-greedy strategy) is linearly decayed over the training epochs. This ensures that the agent initially explores the large state space of Tetris before converging toward the learned optimal policy π_*.

3.4 State Representation (S)

The state space is handled by the `get_state_properties` method in `tetris.py`, offering two distinct representations based on the chosen architecture.

The obsvertaion fed to the CNN is a 20×10 binary array where 1 represents the presence of a block, and 0 represents an empty space. The hope is that the CNN can discern patterns by itself given this raw board state representation.

For the standard NN, the state representation is simplified to 4 heuristic features:

- **Lines Cleared**: number of lines cleared as a result of the given action.
- **Height**: Sum of the height (highest points with a filled square) of all columns
- **Bumpiness**: Sum of the absolute differences in height between adjacent columns.
- **Holes**: Number of empty spaces with a filled space somewhere above it.

This simplified domain knowledge has proven to be effective in training linear layers for this Tetris task [SP16].

4 Experiments and Results

4.1 Comparison of state representations

To begin with, both the raw board CNN and heuristic NN models were given the same reward function for training.

The first obvious difference between the 2 was the amount of epochs needed for the models to understand the game. **JAKOB CREATE GRAPHICS TO COMPARE MODELS**

In terms of behavior, the raw CNN model starts every game by placing blocks at random with no evident strategy. Only when it gets close to losing does it start placing blocks strategically, prolonging the game further.

The model fails to understand that the placement of the earlier pieces affects the score. Furthermore, it cannot take advantage of the exponential scoring of clearing multiple lines and does not seem to aim for Tetrises.

On the other hand, the heuristic NN model is very efficient in its block placement from the very beginning. Frequently, it leaves an entire column (most often the leftmost or right most column) empty so that it can fit a line piece to score a Tetris.

However, the model sometimes over-focuses on achieving Tetrises, and ends up losing the game before getting the line piece in. A such, it sometimes prioritizes the exponential scoring in the short term over the increase in scoring from prolonging the game.

4.2 Variations in given Rewards

References

[Gee25] GeeksforGeeks. *Q-Learning in Reinforcement Learning*. Accessed: 2026-05-07. 2025. URL: https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/.

[Ngu03] Viet Nguyen. *Tetris-deep-Q-learning-pytorch*. Accessed: 2026-05-03. 2023, 04, 03. URL: https://github.com/vietnh1009/Tetris-deep-Q-learning-pytorch.

[SB18] Richard S. Sutton and Andrew G. Barto. "Reinforcement Learning: An Introduction". In: 2nd ed. Cambridge, MA: MIT Press, 2018.

[SP16] Matt Stevens and Sabeek Pradhan. *Playing Tetris with Deep Reinforcement Learning*. Tech. rep. Stanford University, 2016. URL: https://cs229.stanford.edu/proj2016/report/StevensPradhan-PlayingTetrisWithDeepReinforcementLearning-report.pdf.